

Local Todo App Documentation

Generated from the current repository structure and codebase on 2026-08-11.

1. Overview

This repository contains a local-first todo application built with Next.js and SQLite. The UI is served from a Next.js app, and task persistence is handled by a native SQLite database stored on disk.

Core capabilities include creating tasks, editing them, archiving them, filtering by topic/status, sorting by due date, and retaining task history without deleting records.

2. Application architecture

- Frontend: Next.js 14.2.5 with React 18.3.1 and React DOM 18.3.1.
- API layer: route handlers in `pages/api/tasks.js` for GET, POST, PUT, and PATCH requests.
- Persistence layer: `better-sqlite3` 8.3.0 manages the SQLite database connection and SQL statements.
- Testing layer: Jest 29.7.0 with Babel and Testing Library support for behavior verification.

3. Database structure

The application uses a single table named `tasks`. The database file is created under the `data` folder by default, and the location can be overridden with the `TASKS_DB_PATH` environment variable.

Table: `tasks`

- `id`: INTEGER PRIMARY KEY AUTOINCREMENT
- `title`: TEXT NOT NULL
- `description`: TEXT
- `dueDate`: TEXT NOT NULL
- `topic`: TEXT NOT NULL
- `status`: TEXT NOT NULL (allowed values: `Todo`, `In-Progress`, `Complete`)
- `archived`: INTEGER NOT NULL DEFAULT 0
- `createdAt`: TEXT NOT NULL
- `updatedAt`: TEXT NOT NULL

The database layer in `lib/db.js` creates the table if it does not exist, serializes rows into task objects, and provides CRUD operations for reading, creating, updating, and archiving tasks.

4. AI versions and model usage

No AI model, LLM SDK, or AI service integration is currently implemented in this repository. The application is a standard Next.js + SQLite todo app rather than an AI-powered product.

Because there is no AI integration, there are no AI model versions such as GPT, Gemini, Claude, or OpenAI SDK versions to report from the package manifest.

5. Tests involved

The current test suite contains two Jest test files:

- `__tests__/api.test.js`: verifies the API route end-to-end for creating tasks, listing tasks, and archiving tasks.
- `__tests__/db.test.js`: verifies the database module directly for creating tasks, updating task status, and archiving tasks.

Detailed test coverage:

- API create-and-read flow: creates a task through POST and confirms it appears in GET results.
- API archive flow: creates a task, archives it with PATCH, and confirms it appears in archived results.
- Database create flow: verifies a newly created task includes the required fields and appears in the active task list.
- Database update flow: creates a task, updates its status, and confirms the status filter returns the updated record.
- Database archive flow: creates a task, archives it, and confirms it is hidden from active results but visible in archived results.

Verification result from the current environment: 2 test suites passed, 5 tests passed, 0 failed.

6. How to run the project and tests

Install dependencies with `npm install`, start the app with `npm run dev`, and run tests with `npm test` (or the local Jest binary).